THOMAS MORE

IQ Noodles Puzzle Assistant

Realization Document

Abdul Salam Aldabik
Student Bachelor Applied Computer Science

2025-2026

Table of Contents

1. Introduction

This document describes the realization of the IQ Noodles puzzle assistant, the system built during an internship for Smart NV. The project plan and project charter set out the wider vision, the business goals and the planning; this document picks up from there and explains what was actually designed, trained, built and tested. The chapters move from the analysis behind the main technical choices, through the realized result, to the testing and the conclusion. The reflection on the internship is delivered as a separate document.

1.1. Project overview

Smart NV, which sells its games under the SmartGames brand, makes physical logic puzzles for children and families. In IQ Noodles, the player fills a small board with eleven uniquely shaped, colour-coded "noodle" pieces (SmartGames, 2024). A player who gets stuck has only the printed solution booklet, which needs reading and gives away the whole answer instead of a small hint. The internship looked at how computer vision and machine learning could offer that smaller hint. The result is a mobile-first Progressive Web App: the player holds a phone above the board, taps one button, and the app reads the board from a photo and suggests the next piece or a full solution. Figure 1 shows a physical board, the only input the system receives.

Figure 1. A physical IQ Noodles board photographed with a smartphone. The curved pieces thread around the twenty-one pins. This raw photo is the only input to the system.

1.2. Objectives

The project charter set four objectives for the IQ Noodles subsystem, which this document treats as the targets the result is measured against.

- **Detect game state.** Read from one photo which of the eleven pieces are on the board and where.
- **Compute a solution.** Find a valid completion of the current board in real time.
- **Provide hints.** Give age-appropriate guidance without making the player read a booklet.
- **Respect privacy.** Make sure photos of children are never transmitted or stored.

These objectives, together with the non-functional requirements in Table 1, are inherited from the project plan and drive the analysis in the next chapter.

Table 1. Binding non-functional requirements inherited from the project plan.

Requirement	Definition adopted for the result
Privacy (GDPR)	No photo of a child leaves the device; only abstract board state is used.
Offline capability	The application must work without a network connection.
Cross-browser support	Must run in mobile Safari (iOS) and Chrome (Android).
Performance	Detection and solving must feel real-time on mid-range mobile hardware.
Future-proofing	Architecture must allow on-device machine-learning inference.

1.3. Context and timeframe

The internship ran during the 2025-2026 academic year, from 23 February to 22 May 2026, on site at Smart NV as a research-phase initiative. Several parties have a stake in the result. The client, Smart NV, provides the puzzle assets and the requirements and accepts the final result. The end users are children from about age eight and the parent or teacher beside them. The development team carried out the work; within it, the author was responsible for the full IQ Noodles subsystem described here, while other members handled the company's other games. The system itself runs on the player's own phone, in the browser, which is where all detection and solving take place.

The work followed the phased schedule set out in the project plan, summarised below.

- **Foundation (weeks one to two):** onboarding, requirements and the agreed approach.
- **Data and first model (weeks two to five):** synthetic data generation and a first trained model.
- **Engine and application (weeks five to eight):** the puzzle engine, the solver and the application skeleton.
- **Vision and fine-tuning (weeks eight to eleven):** the on-device pipeline, two-phase training and scan mode.
- **Validation and handover (weeks eleven to thirteen):** real-world testing, polish and the portfolio documents.

The chapters that follow describe the analysis behind the main choices, the realized result and its testing, and the conclusion looks back at the objectives stated above.

1.4. Structure of this document

The analysis chapter sets out the main design choices and justifies them, using a weighted ranking for the two decisions that carried the most weight. The three chapters after it describe the realized result: the puzzle engine and solver, the computer-vision side covering data, training and the on-device pipeline, and the application itself. A testing chapter reports the experiments and results, and the conclusion looks back at the objectives. Six appendices (A to F) hold the heavier detail, and each is referenced from the main text.

2. Analysis

This chapter explains why the system is built the way it is. Each section states a problem, weighs the options, and gives the reasoning for the choice. For the two decisions with the largest impact, the on-device versus cloud question and the three-dimensional rendering engine, the reasoning is made explicit with a weighted ranking, where each option is scored from one (poor) to five (excellent) against weighted criteria and the weighted totals are compared.

2.1. Representing the board

IQ Noodles is not a grid puzzle. Its pieces are curved paths that link the pins of the board rather than rigid blocks, so the board cannot be treated as a plain grid of full or empty cells and a piece cannot be described by a bounding box. Table 2 sets the two puzzle families side by side.

Table 2. Structural comparison of IQ Noodles with a grid-based puzzle.

Property	IQ Noodles	IQ Puzzler Pro
Board type	Pin-based graph (node-link)	Grid (cell-based)
Pieces	Curved noodles linking pins	Polyominoes (rigid blocks)
Constraint	Pieces follow orthogonal paths between pins	Pieces occupy cells without overlap

Three representations were considered. A direct cell-list throws away the pin topology that decides whether a curved piece fits. A segment encoding blows up the state space and makes constraint checking awkward. A pin-based graph was chosen: a fourteen-by-fourteen board with twenty-one pins, each surrounded by four cells, and each piece a sequence of typed segments linking adjacent cells around those pins. This mirrors the physical board and keeps constraint checking simple. The encoding is described in the puzzle-engine chapter and listed in Appendix A (Table 7).

2.2. On-device versus cloud computer vision

The project plan first assumed photos would be uploaded to a server, processed there, and pushed to a separate dashboard. Running everything on the device was the alternative. Because this choice touches privacy, offline use and cost all at once, it was scored with a weighted ranking in Table 3.

Table 3. Weighted ranking of on-device versus cloud processing (scores from one to five; weighted totals in the last row).

Criterion	Weight	On-device	Cloud
Privacy and GDPR fit	0.30	5	2
Offline capability	0.20	5	1
Responsiveness (no network round-trip)	0.20	4	3
Infrastructure cost and simplicity	0.15	5	2
Implementation effort in the browser	0.15	2	4
Weighted total	1.00	4.45	2.25

On-device processing wins clearly. It keeps photos on the phone, which settles the GDPR question for images of children, works offline, and removes server cost and latency. Its one weakness is implementation effort, because every stage after the model, from perspective correction to colour analysis, had to be written by hand in TypeScript: the browser has no equivalent of OpenCV. That cost was judged acceptable against the privacy and offline gains.

2.3. Identifying pieces: masks versus colour

A second vision decision was how to tell the eleven pieces apart. The first approach used the YOLO segmentation masks alone (Redmon & Farhadi, 2018). It failed when pieces were tightly packed, because a mask leaked from one noodle into the next and the dark board showed through the hollow tubes. A colour-based classifier working in the perceptually uniform CIE L*a*b* space (CIE, 2004) became the primary signal, with the YOLO geometry kept as a spatial booster. The realized pipeline and the measured effect are covered in the computer-vision and testing chapters.

2.4. Solver algorithm: backtracking versus exact cover

Two solvers were weighed. Knuth's Algorithm X with Dancing Links is a strong exact-cover solver, but it is harder to debug and supports partial boards and hints less naturally, which matters because the app needs hints from any partial board. A backtracking search with a Minimum Remaining Values (MRV) heuristic was chosen instead (Russell & Norvig, 2020). It picks the most constrained free cell first and prunes dead ends early. The first version without a heuristic took about twenty-four seconds on a full board; the cell-level MRV heuristic brought that down to roughly thirty milliseconds, which settled the choice. Algorithm X stays on file if a larger Smart NV board ever needs it.

2.5. Synthetic data: PyRender versus Blender

No labelled dataset existed, so training data had to be generated. The first generator pulled piece geometry from the company's STL files with Trimesh and rendered it with PyRender. PyRender could not produce realistic lighting or surface detail, and the renders looked little like the real pieces. The generator was rebuilt in Blender, which renders at much higher quality, is already used in the department, and exports segmentation masks automatically. The realized dataset is described in the computer-vision chapter.

2.6. Technology stack

The remaining choices were made against the cross-browser and performance requirements. React 19 with Vite and TypeScript gives modular, typed, testable components with fast builds (Vite, 2024). ONNX Runtime Web runs the model in the browser with a WebGPU path and a WebAssembly fallback (Microsoft, 2024). The detection model is YOLO26n-seg, the nano segmentation variant, chosen for inference speed over the small accuracy gain of a heavier model (Ultralytics, 2024). For three-dimensional rendering, React Three Fiber (Pmndrs, 2024) and Babylon.js were the two candidates, scored in Table 4.

Table 4. Weighted ranking of the 3D rendering engine (scores from one to five; weighted totals in the last row).

Criterion	Weight	React Three Fiber	Babylon.js
Integration with the React code	0.30	5	3
Performance on a phone	0.25	4	4
WebGL context efficiency (shared context)	0.20	5	3
Ecosystem and learning curve	0.15	4	3
OBJ model loading	0.10	4	4
Weighted total	1.00	4.55	3.35

React Three Fiber was chosen. It fits the existing React code naturally and renders the eleven piece previews through one shared WebGL context, which keeps the app inside the eight-context limit of mobile Safari. An earlier Babylon.js prototype was set aside.

3. Puzzle engine and solver

This chapter describes the engine that models the board and the solver that completes it. Figure 2 shows where these parts sit in the full system.

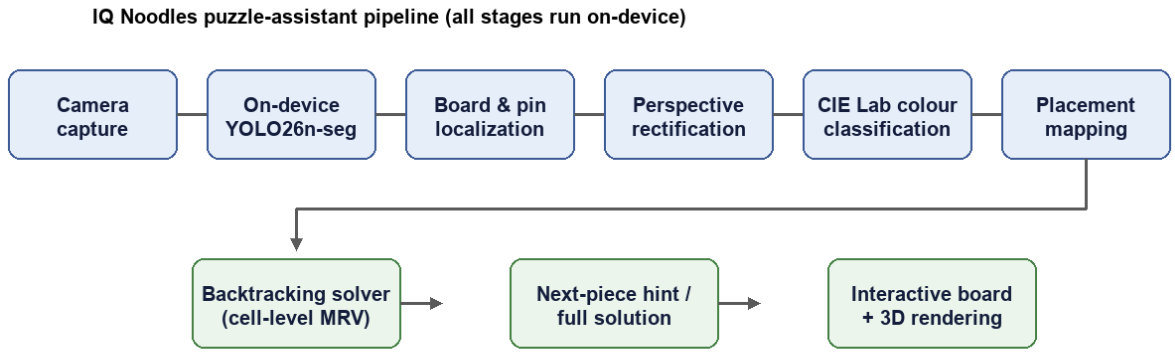

Figure 2. End-to-end architecture. A camera frame passes through on-device detection, localization, rectification, colour classification and placement mapping; the resulting board state then feeds the solver, the hints and the interface. Every stage runs on the device.

3.1. Board representation

The board is a flat fourteen-by-fourteen grid addressed by a single integer index. Of its hundred and ninety-six positions, a hundred and twelve are absent because of the diamond-shaped cut-outs, which leaves exactly eighty-four playable cells, a count the test suite asserts. The twenty-one pins are stored as the four cells around each of them, with a constant-time reverse lookup from any cell to its pin. A useful correction during the project was realising that the pins themselves are not the placement reference: it is the four cells around each pin that decide where a piece connects.

3.2. Pieces and orientations

All eleven pieces are defined as sequences of typed segments (CURVE, CROSS_NS, CROSS_EW). The legal placement rules, where a piece may sit only on free cells fully inside the board, came from a Java reference the company provided. Orientation is handled in an auxiliary design grid: each piece can be rotated through four right angles and mirrored, giving up to eight candidates, after which geometrically identical orientations are dropped. The full piece definitions are in Appendix A (Table 7).

3.3. Backtracking solver, validation and hints

Before solving, every legal placement of every piece is computed once. The solver is a backtracking search with the cell-level MRV heuristic from the analysis chapter (Russell & Norvig, 2020): it picks the free cell with the fewest covering options, prunes any cell with no option, and respects a timeout. Two features sit on top of it. Validation reports whether a partial board is still solvable, returning "solution exists", "dead end" or "pieces overlap". The hint function returns the next piece to place. Both answer in tens of milliseconds. The full algorithm and timings are in Appendix B.

4. Computer vision: data, training and pipeline

This chapter covers how the model was given data, how it was trained, and how the on-device pipeline turns a photo into a board state.

4.1. Synthetic data generation

Each synthetic image is a Blender render of the board from a randomised overhead camera, with randomised piece colours, poses, lighting and background, and Blender exports the segmentation masks with it, so this phase needs no manual annotation. Figure 3 shows representative pieces.

Figure 3. Synthetic IQ Noodles pieces rendered in Blender, each labelled automatically. Colour, pose, lighting and background are randomised so a model trained mostly on synthetic data still transfers to real photos.

The dataset was structured on purpose to avoid two failure modes seen in early runs: a model that favours over-represented pieces, and a model that overfits to one background. About two thousand images were generated with each piece appearing both alone and in clusters of growing size, roughly a quarter using the real board as background and the rest using randomised indoor HDRI environments. After an early redesign, the board outline and the physical hinge were added as their own classes, the board mask giving a precise border reference and the hinge giving a reliable orientation anchor.

4.2. Two-phase model training

The model is YOLO26n-seg, about 2.7 million parameters (Ultralytics, 2024). Training runs in two phases of transfer learning to get around the shortage of real annotated photos. Phase 1 trains on the synthetic dataset and teaches the shapes of the pieces, the board and the pins. Phase 2 fine-tunes those weights on a small set of real, hand-annotated photos, with the earliest layers frozen so the synthetic features survive. The real photos were annotated on Roboflow (Roboflow, 2026); because they had no pin labels at first, the Phase 1 model pre-labelled the pins automatically and those were corrected by hand. The exported model is a 10.6 MB ONNX file the browser downloads once and caches. The experimental set-up is in the testing chapter and the hyperparameters in Appendix B (Tables 8 and 9).

4.3. Vision pipeline

The pipeline turns one camera frame into a list of piece placements. It runs entirely in the browser and, apart from the ONNX model, is written by hand with no external vision library. It has five stages in sequence.

1. On-device inference: the model runs on the letterboxed frame and its outputs are decoded into detections and masks.
2. Board and pin localization: the board outline becomes a four-corner quadrilateral and the pins become sub-pixel centroids.
3. Perspective correction: a homography fitted from the detected pins warps the frame into a top-down view, more accurate than using the corners alone.
4. Colour classification: each of the eighty-four cells is classified in CIE L*a*b* space as empty, pin, or one of the eleven pieces.
5. Placement mapping: the colour evidence and the YOLO geometry are merged into one conflict-free board state.

Each stage makes a deliberate robustness choice: a pin-anchored homography instead of a corner-only one, colour classification instead of mask-only detection, and a fallback chain when a preferred method fails. The full mathematics, including the least-squares homography (Hartley & Zisserman, 2004), is in Appendix C, and the tuning constants are in Appendix E (Table 11).

5. Application and user interface

This chapter describes the application the player uses. It is one React component tree built around a single canonical board state. On a phone it opens in scan mode; on a desktop it opens in manual mode, where the player places pieces by hand to get hints.

5.1. Manual mode and 3D rendering

The board is drawn as three stacked layers: an SVG layer for the board, pins and previews; a React Three Fiber layer that lays the three-dimensional piece models pixel-accurately on top; and an invisible layer that captures touch input. The camera is orthographic and top-down rather than perspective, so the board reads as a flat top view and piece positions are not distorted. A control bar offers rotate, flip, clear, validate, solve and hint. The eleven piece previews share one scissored WebGL context so they do not exhaust the mobile-Safari limit, and each model is calibrated through a small per-piece tuning configuration because the exported models do not share a common native orientation.

5.2. Scan mode and camera

In scan mode the camera view adds framing guides and a translucent board outline to steer the player toward the hinge-up orientation the localizer expects. After a scan the app shows how many pieces were found and lets the player apply the result or retake the photo. Figure 4 shows a real board in an orientation typical of a mid-game scan, with the strong highlights on the glossy plastic that made colour robustness necessary. The interface is mobile-first throughout, with a full-bleed layout, safe-area insets for notched devices and large one-thumb controls.

Figure 4. A second real board configuration. The interface has to handle pieces in many orientations and the strong specular highlights on the glossy plastic.

6. Testing and results

This chapter covers how the system was tested, the conditions under which the model was evaluated, and the measured results.

6.1. Test strategy

Twelve test files run under Vitest and cover the parts that work in a headless environment: the engine (board constants, segment lengths, orientation deduplication), the solver (placement round-trips, clone independence, timeouts), the coordinate system, the YOLO post-processing, and the vision geometry (localization, the pin homography and its inlier refinement, rectification). The browser-only components, the colour classifier, the placement mapper, the camera view and the rendering layer, cannot be unit-tested without heavy mocking and were validated through real-world testing instead. That limit is stated openly rather than hidden.

6.2. Experimental set-up

The model was evaluated under controlled conditions so the results are reproducible and comparable across phases.

- **Hardware.** A single NVIDIA RTX 3060 Ti GPU (8 GB VRAM), CUDA 12.8, PyTorch 2.11.0, Ultralytics 8.4.39. Training ran on a self-hosted machine with persistent remote sessions to avoid cloud time limits.
- **Synthetic dataset (Phase 1).** About two thousand Blender images, roughly 1 600 training and 480 validation. The validation set holds 5 863 annotations, with twenty-one pins per image.
- **Real dataset (Phase 2).** 101 smartphone photos annotated on Roboflow. After filtering, an eighty-twenty split with a fixed seed gave eighty-one training and twenty validation images.
- **Annotation discipline.** For every image either all twenty-one pins are labelled or none are, so the model never learns to read an unlabelled pin hole as background.
- **Metrics.** Mean Average Precision at an intersection-over-union of 0.5 (mAP@0.5) and averaged over 0.5 to 0.95, for boxes and masks, plus inference time per image.

Figure 5 documents the synthetic dataset. The per-class balance on the left confirms no piece is over-represented, and the object-count distribution on the right shows the intended mix of single pieces and clusters.

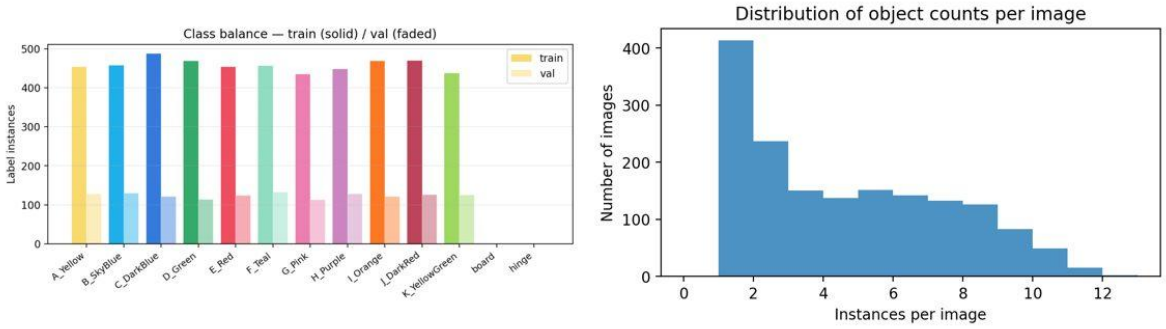

Figure 5. Synthetic dataset composition. Left: label instances per class for the train and validation splits. Right: number of images by object count.

6.3. Results

Each phase was evaluated on its own validation set. Table 5 puts the headline metrics side by side.

Table 5. Validation results for Phase 1 (480 synthetic images) and Phase 2 (20 real photos).

Metric	Phase 1 (synthetic)	Phase 2 (real)
Box mAP@0.5	0.9919	0.9402
Box mAP@0.5:0.95	0.9701	0.8613
Mask mAP@0.5	0.9875	0.9403
Mask mAP@0.5:0.95	0.9264	0.7839
Inference speed (RTX 3060 Ti)	3.5 ms	24.2 ms

Phase 1 reaches near-ceiling accuracy on synthetic data, which confirms the model capacity and the synthetic labels are sound. The drop after Phase 2, from 0.9875 to 0.9403 on mask mAP@0.5, is expected and is not a regression. The real validation set is harder, with variable lighting, reflections and shadows the renders only approximate, and much smaller at twenty images, so each image carries a lot of weight. A mask mAP@0.5 of 0.94 on unconstrained phone photos is strong for a 2.7-million-parameter model running in a browser. Table 6 shows the strongest and weakest classes; the full per-class table is in Appendix D (Table 10).

Table 6. Best- and weakest-performing piece classes on the real validation set (mask metrics).

Class	Precision	Recall	mAP@0.5
G_Pink (best)	0.850	1.000	0.995
E_Red	1.000	0.962	0.995
J_DarkRed	0.930	1.000	0.995
C_DarkBlue	0.743	0.827	0.825
K_YellowGreen (weakest)	1.000	0.702	0.715

The weakest classes, K_YellowGreen and C_DarkBlue, are the ones whose colour is easiest to confuse with the dark board or with shadow. That is the empirical reason the colour classifier was added in CIE L*a*b* space (CIE, 2004), where it does not lean on the YOLO class prediction for these difficult pieces. The design decision and the measured weakness line up directly.

6.4. Real-world scan validation

More than sixty labelled capture sessions were recorded under indoor light, natural light, direct sunlight and shadow. Each saved the photo together with annotated overlays of every pipeline stage, and a developer page let images be scanned in batches and every stage inspected with live controls. Inspecting these overlays was the main way the colour thresholds, white-balance anchors and homography quality gates were tuned. A set of quality gates rejects scans that are too oblique or poorly aligned and asks for a retake rather than quietly returning a wrong board state.
The telemetry from a single representative capture shows how the quality gates behave on a good scan. From a 2252 by 4000 photo the model returned 34 detections: the eleven pieces, the board and 22 pins. All 21 board pins matched their canonical positions with a mean grid residual of 0.06 cells, far inside the 0.22-cell retry threshold, and the homography condition number was 1.4, far below the 5 000 reject gate. Every one of the eleven pieces was placed, at an average confidence of 92 per cent. Figure 6 shows the raw model detections for this scan.

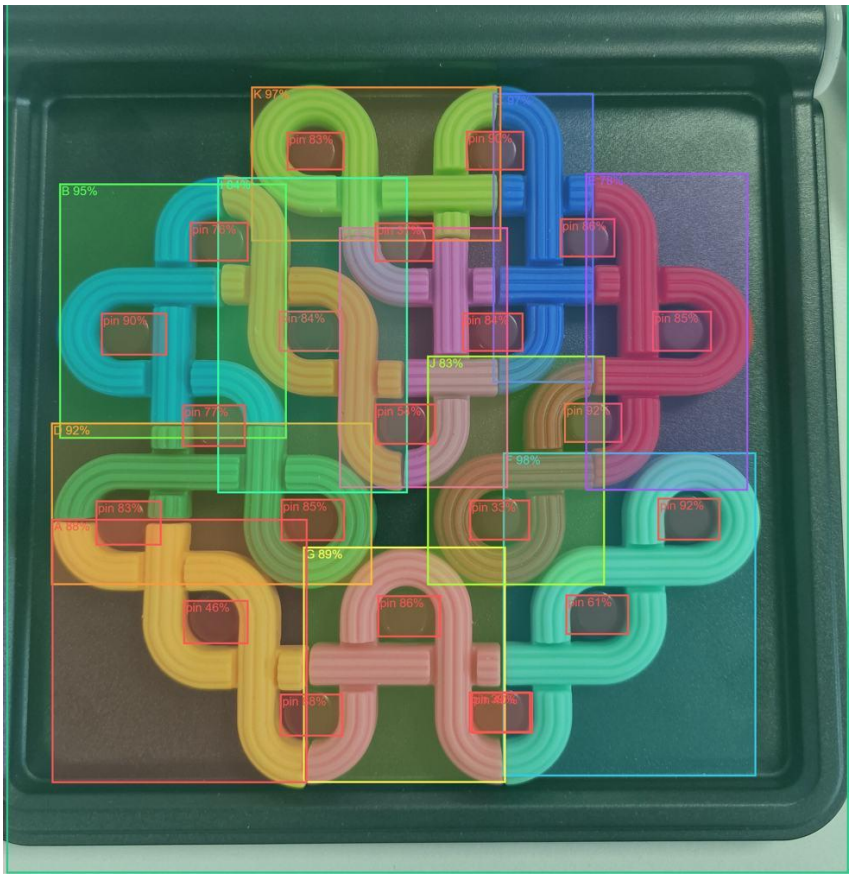

Figure 6. Raw YOLO26n-seg detections on a real photo, with a bounding box and confidence for each of the eleven pieces, the board and the 22 detected pins. This on-device model output feeds the rest of the pipeline.

Figure 7 shows one of these captures taken through the scan-lab tool, from the raw board localization to the final mapped board state.

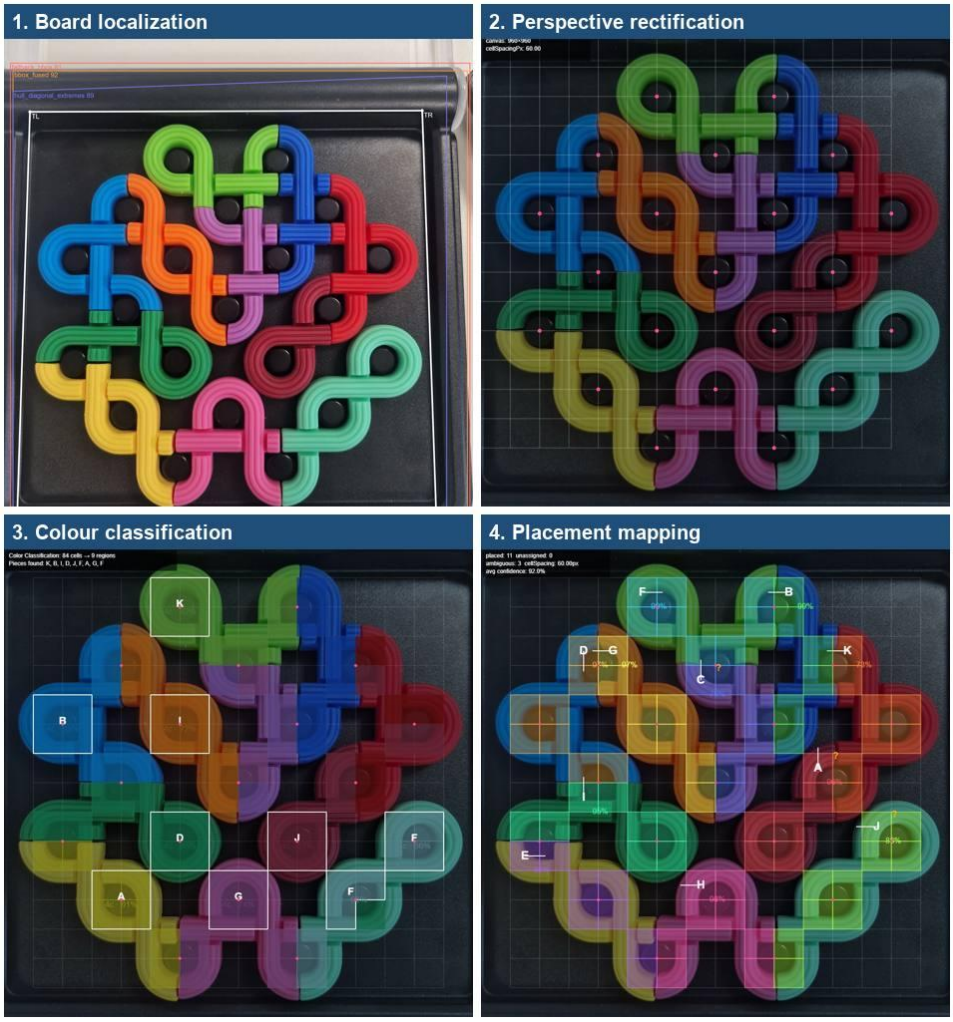

Figure 7. A real scan processed through the scan-lab developer tool, shown in four stages: board localization, perspective rectification to a 960 by 960 top-down view, colour classification of the 84 cells, and the final placement mapping with all eleven pieces placed at an average confidence of 92 per cent.

7. Conclusion

The IQ Noodles puzzle assistant was designed, trained, built and validated as a complete, on-device, mobile-first Progressive Web App, and it meets the four objectives from the introduction. It reads the board from a single photo, computes a solution with a backtracking solver that clears an empty board in tens of milliseconds, and offers next-piece hints, all without any photo leaving the device, which satisfies the privacy and offline requirements. The two-phase training reaches a mask mAP@0.5 of 0.99 on synthetic data and 0.94 on real photos, and the weighted-ranking analysis in the analysis chapter shows why the on-device and React Three Fiber choices were the right ones for these requirements.
A few parts are left for later and are listed here so the result above is read as a description only of what is finished. Collecting more real photos of the weaker classes would narrow the gap between synthetic and real accuracy. The browser-only components could be covered by a headless-browser test harness. The modular architecture also makes it feasible to retrain the same pipeline for other Smart NV games, and a user study with children in the target age group would test hint quality and the scan flow, which this technical phase did not formally measure.

Reference list

CIE. (2004). Colorimetry (3rd ed.). CIE 015:2004. Commission Internationale de l'Eclairage.

Hartley, R., & Zisserman, A. (2004). Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press.

Microsoft. (2024). ONNX Runtime Web (version 1.24.3) [Software]. https://onnxruntime.ai/

Pmndrs. (2024). React Three Fiber (version 9) [Software]. https://docs.pmnd.rs/react-three-fiber/

Redmon, J., & Farhadi, A. (2018). YOLOv3: An Incremental Improvement. arXiv:1804.02767.

Roboflow. (2026). noodels dataset (version 4) [Data set]. https://roboflow.com/

Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

SmartGames. (2024). IQ Noodles puzzle [Product page]. https://www.smartgames.eu/uk/one-player-games/iq-noodles

Ultralytics. (2024). YOLO documentation: Instance segmentation. https://docs.ultralytics.com/tasks/segment/

Vite. (2024). Vite documentation. https://vitejs.dev/

Attachments

The appendices below hold the implementation detail that supports the main text without being needed to follow it. Each one is referenced from the relevant chapter, as noted in the analysis, puzzle-engine, computer-vision and testing chapters.

Appendix A: Piece definitions

Referenced from the analysis and puzzle-engine chapters. Table 7 lists all eleven pieces, with key, segment count and topology.

Table 7. The eleven IQ Noodles pieces, with key, segment count and segment topology.

ID	Key	Segments	Topology
0	J (Dark Red)	6	CURVE, CROSS_EW, CURVE, CROSS_EW, CURVE, CURVE
1	C (Dark Blue)	6	CURVE, CROSS_NS, CROSS_EW, CURVE, CROSS_NS, CROSS_EW
2	H (Purple)	6	CURVE, CROSS_NS, CROSS_NS, CROSS_EW, CURVE, CROSS_EW
3	B (Sky Blue)	8	CURVE, CROSS_NS, CROSS_NS, CURVE, CROSS_EW, CURVE, CURVE, CROSS_EW
4	A (Yellow)	8	CROSS_EW, CURVE, CURVE, CROSS_EW, CROSS_EW, CURVE, CROSS_EW, CURVE
5	K (Yellow-Green)	8	CURVE, CURVE, CURVE, CROSS_NS, CROSS_NS, CROSS_EW, CURVE, CROSS_EW
6	I (Orange)	8	CROSS_EW, CURVE, CURVE, CROSS_EW, CROSS_EW, CURVE, CURVE, CROSS_EW
7	G (Pink)	8	CURVE, CROSS_NS, CROSS_NS, CURVE, CROSS_EW, CURVE, CURVE, CROSS_EW
8	D (Green)	8	CURVE, CURVE, CURVE, CROSS_NS, CROSS_EW, CURVE, CROSS_NS, CROSS_EW
9	F (Teal)	10	CURVE, CROSS_NS, CROSS_NS, CURVE, CURVE, CROSS_NS, CROSS_NS, CURVE, CURVE, CURVE
10	E (Red)	8	CURVE, CROSS_NS, CROSS_EW, CURVE, CROSS_NS, CROSS_EW, CURVE, CURVE

Appendix B: Solver internals and hyperparameters

Referenced from the analysis, puzzle-engine and computer-vision chapters. The solver builds a cell-coverage index mapping each free cell to the (piece, placement) pairs that can cover it, picks the cell with the fewest options, prunes zero-option cells, and stops early on a forced cell. It checks a deadline once per recursive call (5 000 ms to solve, 2 000 ms to validate). This heuristic cut full-board solve time from about twenty-four seconds to roughly thirty milliseconds against naive backtracking. Tables 8 and 9 give the training hyperparameters.

Table 8. Phase 1 training hyperparameters (synthetic data).

Hyperparameter	Value
Epochs	100 (early-stop patience 20)
Batch size	16
Image size	640 x 640
Optimiser	AdamW (Ultralytics default)
Initial learning rate	about 0.01
Augmentation	HSV jitter, rotation +/-15 deg, scale 0.4, fliplr 0.5, mosaic 0.8, mixup 0.1, copy-paste 0.2

Table 9. Phase 2 fine-tuning hyperparameters (real data).

Hyperparameter	Value
Epochs	50 (early-stop patience 15)
Batch size	16
Image size	640 x 640
Initial learning rate	0.001 (ten times lower than Phase 1)
Frozen layers	First five
Augmentation	Reduced: HSV jitter, rotation +/-10 deg, scale 0.3, mosaic 0.5, mixup 0.05

Appendix C: Vision-pipeline mathematics

Referenced from the computer-vision chapter. The production code writes each of these from scratch in TypeScript; the multi-view geometry follows standard methods (Hartley & Zisserman, 2004).

- **Board localization.** The board mask is reduced to a convex hull (Andrew's monotone chain), and three candidate quadrilaterals are scored on rectangularity (0.40), aspect ratio (0.25), side balance (0.20) and hinge alignment (0.15).
- **Pin-anchored homography.** Each pin correspondence adds two rows to an over-determined system solved by least squares (normal equations, Gaussian elimination with partial pivoting), with one inlier-refinement pass that drops residuals over half a cell.
- **Degeneracy check.** The condition number of the homography is found analytically from the eigenvalues of H-transpose-H via Cardano's method; above 5 000 the scan is rejected as too oblique.
- **Perspective rectification.** The frame is warped into a 960 x 960 view by backward mapping with bilinear interpolation (60 pixels per cell).
- **Colour classification.** The median cell colour is converted from sRGB to linear to XYZ to CIE L*a*b*, then matched to thirteen calibrated references by Euclidean delta-E through a five-rule tree.

Appendix D: Per-class Phase 2 validation results

Referenced from the testing chapter. Table 10 gives the full per-class results on the twenty real validation photos.

Table 10. Per-class Phase 2 validation results on the twenty real photos.

Class	Box P	Box R	Box mAP@0.5	Mask mAP@0.5	Mask mAP@0.5:0.95
A_Yellow	0.811	0.864	0.962	0.962	0.799
B_SkyBlue	0.848	1.000	0.995	0.995	0.771
C_DarkBlue	0.743	0.827	0.825	0.825	0.671
D_Green	0.732	0.686	0.843	0.843	0.651
E_Red	1.000	0.962	0.995	0.995	0.856
F_Teal	1.000	0.889	0.995	0.995	0.861
G_Pink	0.850	1.000	0.995	0.995	0.904
H_Purple	0.733	1.000	0.995	0.995	0.816
I_Orange	0.683	0.800	0.920	0.920	0.856
J_DarkRed	0.930	1.000	0.995	0.995	0.833
K_YellowGreen	1.000	0.702	0.715	0.715	0.604
board	0.977	1.000	0.995	0.995	0.936
pin	0.966	0.981	0.992	0.993	0.635
ALL	n/a	n/a	0.940	0.940	0.784

Appendix E: Key system constants

Referenced from the puzzle-engine and computer-vision chapters. Table 11 lists the main constants.

Table 11. Key system constants used across the engine and the vision pipeline.

Constant	Value	Purpose
Board dimensions	14 x 14	Grid size (196 positions, 84 playable)
Pins	21	Physical pin clusters
Model input	640 px	YOLO input square
NMS confidence / IoU	0.25 / 0.50	Detection filtering
Homography condition limit	5 000	Reject over-oblique scans
Grid-misaligned threshold	0.22 cells	Mean pin residual retry trigger
Max piece distance	45 delta-E	CIE Lab piece-match limit
Scan confidence threshold	0.15	Minimum to include a placement
Solver timeout (solve / validate)	5 000 / 2 000 ms	Search deadlines
Rectified canvas	960 px	60 px per cell

Appendix F: Coordinate spaces

Referenced from the computer-vision chapter. Table 12 lists the six coordinate spaces used across the system, with one conversion authority between them.

Table 12. The six coordinate spaces used across the system.

Space	Units	Origin	Usage
Image pixel	px	Top-left of raw frame	Raw detections, mask polygons
Letterbox pixel	px (640 sq.)	Top-left of ONNX input	YOLO model input/output
Board unit	cells	Centre of top-left valid cell	Homography target, placement grid
Canvas pixel	px (960 sq.)	Top-left of rectified canvas	Colour classifier, overlays
SVG pixel	px (444 sq.)	Top-left of SVG viewBox	Board rendering
World unit	Three.js	Board centre (Y-up)	3D piece positioning